

VTU COMPUTER ORGANIZATION & ARCHITECTURE

MODULE 1

Basic Structure of Computers & Machine Instructions

PART 1: Basic Structure	PART 2: Machine Instructions
Computer Types & Architecture	Numbers & Number Systems
Functional Units	Arithmetic Operations & Characters
Basic Operational Concepts	IEEE Standard Floating Point
Registers (PC, MAR, MDR, IR)	Memory Locations & Addresses
Bus Structures	Memory Operations
Software & OS	Instruction Sequencing
Processor Clock & Performance	Branching & Condition Codes

vtu-easy.com

Your complete VTU study companion

TABLE OF CONTENTS

PART 1: BASIC STRUCTURE OF COMPUTERS

- [1. Basic Concepts & Computer Architecture](#)
- [2. Functional Units](#)
 - [2.1 Input Unit](#)
 - [2.2 Output Unit](#)
 - [2.3 Memory Unit](#)
 - [2.4 Arithmetic Logic Unit \(ALU\)](#)
 - [2.5 Control Unit](#)
- [3. Basic Operational Concepts](#)
 - [3.1 Processor Registers \(PC, MAR, MDR, IR\)](#)
 - [3.2 Steps to Execute an Instruction](#)
- [4. Bus Structure](#)
 - [4.1 Single Bus Structure](#)
 - [4.2 Multiple Bus Structure](#)
 - [4.3 Memory Mapped I/O](#)
- [5. Software](#)
 - [5.1 Operating System](#)
- [6. Performance](#)
 - [6.1 Processor Clock](#)
 - [6.2 Basic Performance Equation](#)
 - [6.3 Clock Rate](#)
 - [6.4 CISC vs RISC](#)
- [7. Solved Problems – Part 1](#)

PART 2: MACHINE INSTRUCTIONS & PROGRAMS

- [8. Numbers, Arithmetic & Characters](#)
 - [8.1 Sign and Magnitude Representation](#)
 - [8.2 One's Complement Representation](#)
 - [8.3 Two's Complement Representation](#)
 - [8.4 Overflow Conditions](#)
- [9. IEEE Standard for Floating Point Numbers](#)
- [10. Memory Locations & Addresses](#)
 - [10.1 Byte Addressability](#)
 - [10.2 Big-Endian & Little-Endian](#)
 - [10.3 Word Alignment](#)
- [11. Memory Operations \(Load & Store\)](#)
- [12. Instructions & Instruction Sequencing](#)
 - [12.1 Register Transfer Notation \(RTN\)](#)
 - [12.2 Basic Instruction Types](#)
 - [12.3 Instruction Execution & Straight Line Sequencing](#)

[12.4 Branching](#)

[12.5 Condition Codes](#)

[13. Solved Problems – Part 2](#)

[14. Quick Revision Summary](#)

PART 1: BASIC STRUCTURE OF COMPUTERS

1. Basic Concepts & Computer Architecture

Computer Architecture (CA) is concerned with the structure and behaviour of the computer. CA includes the information formats, the instruction set, and techniques for addressing memory.

CA covers 3 aspects of computer design:

1. **Computer Hardware:** Consists of electronic circuits, displays, magnetic and optical storage media, and communication facilities.
2. **Instruction Set Architecture (ISA):** The programmer-visible machine interface — instruction set, registers, memory organization, and exception handling. Two main approaches: **CISC** (Complex Instruction Set Computer) and **RISC** (Reduced Instruction Set Computer).
3. **Computer Organization:** Includes the high-level aspects of design such as memory-system, bus-structure, and design of the internal CPU. It refers to the operational units and their interconnections that realize the architectural specifications.

VTU Tip: CA covers Hardware + ISA + Organization. CISC = many complex instructions; RISC = few simple instructions.

2. Functional Units

A computer consists of 5 functionally independent main parts:

- **Input** — Accepts data from the outside world
- **Memory** — Stores programs and data
- **ALU** — Performs arithmetic and logic operations
- **Output** — Sends processed results to the outside world
- **Control Unit** — Generates timing signals to coordinate all units

for I/O devices (10 M)
Q.Explain the operation of computer with neat block diagram (10 M)

FUNCTIONAL UNITS

- A computer consists of 5 functionally independent main parts:

- 1) Input
- 2) Memory
- 3) ALU
- 4) Output &
- 5) Control units.

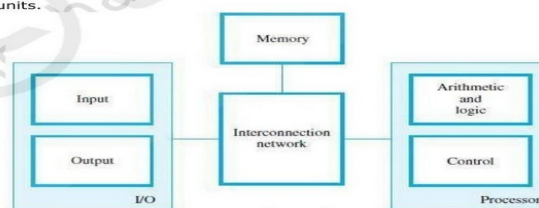


Figure 1.1 Basic functional units of a computer.

Figure 1.1 — Basic Functional Units of a Computer

2.1 Input Unit

Computers accept coded information through input units, which read data. Whenever a key is pressed, the corresponding character or number is translated into its equivalent binary code over a cable and fed either to memory or processor.

Examples: Keyboard, Joystick, Trackball, Mouse, Scanner, Barcode reader, Microphone

2.2 Output Unit

The basic function of the output unit is to send the processed results to the outside world in a human-readable or machine-usable form.

Examples: Printer, Speakers, Monitor, Projector, Plotter

2.3 Memory Unit

The memory unit stores programs and data. It is of two types:

4. **Primary Memory:** Programs which have to be executed **MUST** be stored in primary memory. Each location has a unique address and stores data. **Examples:** RAM, ROM, Cache
5. **Secondary Memory:** Used for large amounts of data and programs that are stored long-term and accessed infrequently. **Examples:** Magnetic disks, Magnetic tapes, Optical disks (CD-ROM, DVD), SSD, Pen drives

*Key Difference: Primary memory is directly accessible by the CPU. Secondary memory is not. Programs **MUST** be loaded into primary memory before execution.*

2.4 Arithmetic Logic Unit (ALU)

The ALU performs:

- **Arithmetic operations:** Addition, Subtraction, Multiplication, Division
- **Logical operations:** NOT, AND, OR, XOR, NAND, NOR

The operands (data) are brought into the ALU from memory. According to the instruction, the operation is performed in the required sequence. Results can be stored back in memory or in registers.

2.5 Control Unit

The Control Unit generates the actual timing signals that govern the transfer of data between the input unit, processor, memory, and output unit. It does **NOT** perform any data processing itself — it only directs and coordinates all operations.

Key Functions:

- Fetches instructions from memory
- Decodes each instruction
- Generates control signals to execute the instruction
- Controls the sequence of operations

3. Basic Operational Concepts

An Instruction consists of 2 parts:

- **Opcode:** Specifies the operation to be performed (e.g., ADD, LOAD, STORE)
- **Operands:** The data on which the operation is performed

[OPCODE] [OPERANDS]

Steps of instruction execution (example: **ADD LOCA, R0**):

6. Step 1: Fetch the instruction from main memory into the processor (into IR).
7. Step 2: Fetch the operand at location LOCA from main memory into the processor.
8. Step 3: Add the memory operand (fetched contents of LOCA) to the contents of register R0.
9. Step 4: Store the result (sum) in R0.

The same can also be written using 2 instructions:

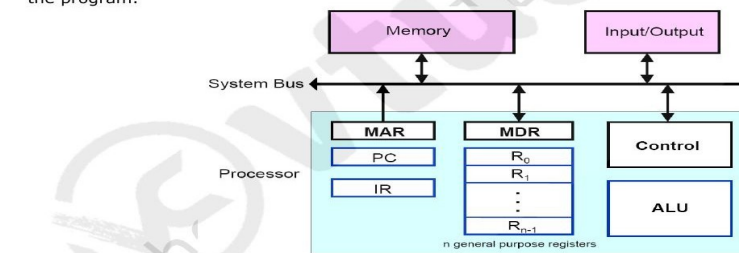
Load LOCA, R1 → Load value at LOCA into register R1
 Add R1, R0 → Add R1 and R0, result in R0

3.1 Processor Registers (PC, MAR, MDR, IR)

The processor contains ALU, control circuitry, and many registers:

Register	Full Name	Function
PC	Program Counter	Contains memory address of the NEXT instruction to be fetched and executed. Updated after each fetch.
IR	Instruction Register	Holds the instruction currently being executed. Decoded by the control unit.
MAR	Memory Address Register	Holds the address of the memory location to be accessed (read or write).
MDR	Memory Data Register	Contains data to be written into or read out of the addressed memory location.
R0–Rn-1	General Purpose Registers	n general-purpose registers available to the programmer for temporary data storage.

5) To fetch an operand, its address is placed into MAR & control-unit issues Read signal. As a result, the operand is transferred from memory into MDR, and then it is transferred from MDR to ALU.
 6) Likewise required number of operands is fetched into processor.
 7) Finally, ALU performs the desired operation.
 8) If the result of this operation is to be stored in the memory, then the result is sent to the MDR.
 9) The address of the location where the result is to be stored is sent to the MAR and a Write cycle is initiated.
 10) At some point during execution, contents of PC are incremented to point to next instruction in the program.



 Q. Explain single bus and multiple bus structure in computer (06 M)

BUS STRUCTURE

- A bus is a group of lines that serves as a connecting path for several devices.
- A bus may be lines or wires.
- The lines carry data or address or control signal.
- There are 2 types of Bus structures: 1) Single Bus Structure and 2) Multiple Bus Structure.

Figure — Processor Registers & Single Bus Connection

3.2 Steps to Execute an Instruction (Between Processor and Memory)

- The address of the first instruction (to be executed) gets loaded into PC.
- The contents of PC (address) are transferred to the MAR & the control unit issues a Read signal to memory.
- After a certain elapsed time, the first instruction is read out of memory and placed into MDR.
- The contents of MDR are transferred to IR. At this point the instruction can be decoded and executed.
- To fetch an operand, its address is placed into MAR & the control unit issues a Read signal. The operand is transferred from memory into MDR, and then from MDR to ALU.
- Likewise, the required number of operands is fetched into the processor.
- Finally, the ALU performs the desired operation.
- If the result is to be stored in memory, the result is sent to the MDR.
- The address of the location where the result is to be stored is sent to the MAR and a Write cycle is initiated.
- At some point during execution, the contents of PC are incremented to point to the next instruction in the program.

VTU Exam Tip: The flow is PC → MAR → Memory Read → MDR → IR → Decode → Execute → Result → MDR → MAR → Memory Write

4. Bus Structure

A Bus is a group of lines/wires that serves as a connecting path for several devices. The lines carry data, address, or control signals.

There are 2 types of Bus Structures:

4.1 Single Bus Structure

In a single bus structure, all devices (CPU, Memory, I/O) are connected to one common bus.

Key Characteristics:

- Only 2 units can actively use the bus at any given time (only one transfer at a time)
- Bus control lines are used to arbitrate multiple requests for bus use
- **Advantage 1:** Low cost
- **Advantage 2:** Flexibility for attaching peripheral devices
- **Disadvantage:** Only one transfer at a time — creates a bottleneck

2) Multiple Bus Structure

- Systems that contain multiple buses achieve more concurrency in operations.
- Two or more transfers can be carried out at the same time.
- **Advantage:** Better performance.
- **Disadvantage:** Increased cost.

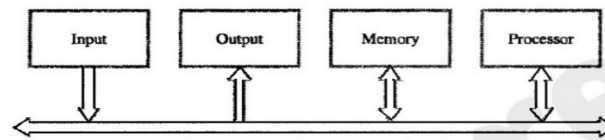


Figure 1.3 Single-bus structure.

- The devices connected to a bus vary widely in their speed of operation.
- To synchronize their operational-speed, buffer-registers can be used.

• Buffer Registers

- are included with the devices to hold the information during transfers.
- prevent a high-speed processor from being locked to a slow I/O device during data transfers.

Memory mapped I/O

Figure 1.3 — Single Bus Structure

4.2 Multiple Bus Structure

Systems that contain multiple buses achieve more concurrency in operations.

- Two or more transfers can be carried out simultaneously
- **Advantage:** Better performance — multiple concurrent transfers
- **Disadvantage:** Increased cost and complexity

Buffer Registers

- Included with devices to hold information during transfers
- Prevent a high-speed processor from being locked to a slow I/O device during data transfers
- Synchronize devices that operate at different speeds

4.3 Memory Mapped I/O

In **Memory Mapped I/O**, some memory address values are used to refer to peripheral device buffer registers (such as DATAIN and DATAOUT). No special I/O instructions are needed — data can be transferred between these registers and the processor using standard instructions like Move, Load or Store.

Example:

MoveByte DATAIN, R1

→ Transfers contents of keyboard buffer DATAIN to register R1

MoveByte R1, DATAOUT

→ Transfers contents of register R1 to output buffer DATAOUT

The status flags SIN and SOUT are automatically cleared when the buffer registers DATAIN and DATAOUT are referenced.

5. Software

System software is a collection of programs that are executed as needed to perform functions such as:

- Receiving and interpreting user commands
- Running standard application programs (word processors, etc.) or games
- Managing the storage and retrieval of files in secondary storage devices
- Controlling I/O units to receive input information and produce output results
- Translating programs from source form (HLL) into object form (machine instructions)
- Linking and running user-written application programs with standard library routines (e.g., numerical computation packages)
- System software is responsible for the coordination of ALL activities in a computing system

5.1 Operating System

Operating System (OS): A large collection of routines that controls the sharing and interaction among various computer units as they perform application programs.

Tasks performed by OS:

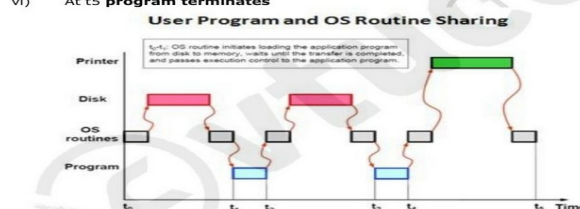
- Assigning memory and magnetic disk space to programs and data files
- Moving data between memory and disk units
- Handling I/O operations
- Scheduling programs for execution

Example sequence (OS + Program execution):

20. **t₀ – t₁:** OS loads the application program from disk to memory
21. **t₁ – t₂:** Program is executed
22. **t₂ – t₃:** Program accesses disk
23. **t₃ – t₄:** Program executes some more
24. **t₄ – t₅:** Program accesses printer
25. **At t₅:** Program terminates

In the diagram shown, the above procedure as per to the time slots is explained.

- i) In the time period $t_0 - t_1$, **Operating system loads the application program from disk to memory.**
- ii) In the time period $t_1 - t_2$ **program is executed**
- iii) $t_2 - t_3$ **program accesses disk**
- iv) $t_3 - t_4$ **program executes some more**
- v) $t_4 - t_5$ **program accesses printer**
- vi) At t_5 **program terminates**



PERFORMANCE

- The most important measure of performance of a computer is how quickly it can execute programs.
- The speed of a computer is affected by the design of
 - 1) Instruction-set.
 - 2) Hardware & the technology in which the hardware is implemented.

Figure — User Program and OS Routine Sharing the Processor

6. Performance

The most important measure of computer performance is how quickly it can execute programs. The speed of a computer is affected by:

26. Design of the instruction set
27. Hardware and the technology in which hardware is implemented
28. Software including the operating system

Since programs are usually written in HLL, performance is also affected by the **compiler** that translates programs into machine language.

For best performance, it is necessary to design the **compiler, machine instruction set, and hardware in a co-ordinated way**.

Role of Cache Memory:

- At the start of execution, all program instructions are stored in main memory
- As execution proceeds, instructions are fetched into the processor, and a copy is placed in the cache
- If the same instruction is needed a second time, it is read directly from the cache (faster!)

- For best performance, it is necessary to design the compiler, machine instruction set and hardware in a co-ordinated way.

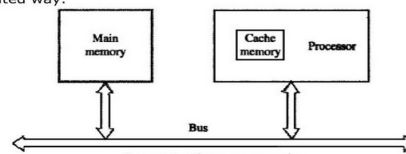


Figure 1.5 The processor cache.

- Let us examine the flow of program instructions and data between the memory & the processor.
- At the start of execution, all program instructions are stored in the main-memory.
- As execution proceeds, instructions are fetched into the processor, and a copy is placed in the cache.
- Later, if the same instruction is needed a second time, it is read directly from the cache.

Q. Explain in brief different types of key parameters that affect the processor performance (05)

Figure 1.5 — The Processor Cache

6.1 Processor Clock

Processor circuits are controlled by a timing signal called a **Clock**. The clock defines regular time intervals called **Clock Cycles**.

- Let **P** = Length of one clock cycle
- Let **R** = Clock rate (cycles per second = Hertz)

$$R = 1/P$$

- To execute a machine instruction, the processor divides the action into a sequence of basic steps
- Each basic step is completed in exactly one clock cycle

Example: Clock rate 1 GHz $\rightarrow P = 1/10^9 = 1$ nanosecond per clock cycle

6.2 Basic Performance Equation

Let:

- **T** = Processor time required to execute a program
- **N** = Actual number of instruction executions
- **S** = Average number of basic steps (clock cycles) needed to execute one machine instruction
- **R** = Clock rate in cycles per second

$$T = (N \times S) / R$$

(Basic Performance Equation)

To achieve HIGH performance (reduce T):

- **Reduce N:** Compile source programs into fewer machine instructions
- **Reduce S:** Use instructions with a smaller number of basic steps

- **Increase R:** Use a higher frequency clock

WARNING: Changes in one parameter often affect others. E.g., increasing R may require simpler instructions (affecting N and S).

6.3 Clock Rate

There are 2 ways to increase the clock rate R:

29. **Improve IC technology:** Makes logic circuits faster, reducing propagation delays
30. **Reduce processing per step:** Reducing the amount of processing done in one basic step reduces the clock period P, allowing a higher rate R

6.4 CISC vs RISC

Feature	RISC	CISC
Instruction complexity	Simple instructions (one cycle)	Complex instructions (multiple cycles)
Control unit	Hardwired control unit	Microprogrammed control unit
Number of instructions	Few instructions	Many instructions
Instruction format	Fixed format	Variable format
Addressing modes	Few (mostly register-to-register)	Many addressing modes
Register set	Multiple register sets	Single register set
Pipelining	Highly pipelined	No or less pipelining
Examples	ARM, MIPS, SPARC	Intel x86, VAX

Pipelining: Overlapping the execution of successive instructions — while one instruction is being executed, the next is being fetched. This improves throughput.

Superscalar: Different instructions are concurrently executed using multiple instruction pipelines. Multiple functional units allow starting execution of several instructions in every clock cycle.

7. Solved Problems – Part 1

These problems are frequently asked in VTU exams. Study the register transfers carefully.

Problem 1: Execute ADD R2, LOCA

List the steps needed to execute: Add R2, LOCA

(Assume the address of the memory location containing this instruction is initially in PC.)

31. **Step 1:** Transfer contents of PC (address of instruction) to MAR.
32. **Step 2:** Issue a Read command to memory and wait until instruction is transferred into MDR.
33. **Step 3:** Transfer the instruction from MDR into IR and decode it.
34. **Step 4:** Transfer the address LOCA from IR to MAR.
35. **Step 5:** Issue a Read command and wait until MDR is loaded with data at LOCA.
36. **Step 6:** Transfer contents of MDR to the ALU.
37. **Step 7:** Transfer contents of R2 to the ALU.
38. **Step 8:** Perform addition of the two operands in ALU and transfer result to LOCA.
39. **Step 9:** Transfer contents of PC to ALU.
40. **Step 10:** Add 1 to operand in ALU and transfer incremented address to PC.

Problem 2: Execute ADD R4, R2, R3

List the steps needed to execute: Add R4, R2, R3

(Assume the address of the memory location containing this instruction is initially in PC.)

41. **Step 1:** Transfer contents of PC to MAR.
42. **Step 2:** Issue a Read command to memory and wait until instruction is transferred into MDR.
43. **Step 3:** Transfer the instruction from MDR into IR and decode it.
44. **Step 4:** Transfer contents of R1 and R2 to the ALU.
45. **Step 5:** Perform addition of two operands in ALU and transfer answer into R3.
46. **Step 6:** Transfer contents of PC to ALU.
47. **Step 7:** Add 1 to operand in ALU and transfer incremented address to PC.

Problem 3: Add contents of A and B, place result in C

(a) Using Load/Store and Add instructions:

```
Load A, R0 → R0 = [A]
Load B, R1 → R1 = [B]
Add R0, R1 → R1 = R0 + R1
Store R1, C → C = R1
```

(b) Using Move and Add (with memory operands) — fewer instructions:

```
Move B, C → C = [B]
Add A, C → C = [A] + [C] = [A] + [B]
```

Problem 4: Performance Calculation

A program contains 1000 instructions: 25% require 4 clock cycles, 40% require 5 clock cycles, and remaining require 3 clock cycles. Clock = 1 GHz.

Solution:

- $N = 1000$, $R = 1 \times 10^9$ Hz
- 25% of 1000 = 250 instructions $\times$ 4 cycles = 1000 cycles
- 40% of 1000 = 400 instructions $\times$ 5 cycles = 2000 cycles
- 35% of 1000 = 350 instructions $\times$ 3 cycles = 1050 cycles

$$T = (250 \times 4 + 400 \times 5 + 350 \times 3) / 10^9 = (1000 + 2000 + 1050) / 10^9 = 4050 / 10^9 =$$

$$T = 4.05 \mu s$$

Problem 5: Performance from Parameters

Clock rate = 800 MHz, Instructions = 1000, Steps/instruction = 20. Find execution time T.

Solution:

$$T = (N \times S) / R = (1000 \times 20) / (800 \times 10^6) = 20000 / 800 \times 10^6$$

$$T = 25 \mu s = 25 \times 10^{-6} \text{ sec}$$

PART 2: MACHINE INSTRUCTIONS & PROGRAMS

8. Numbers, Arithmetic & Characters

Computers are built using logic circuits that operate on information in binary form (0s and 1s). A number in a computer is represented by a string of bits called a **binary number**. A text character is represented by a string of bits called a **character code**.

Numeric Data:

- **Fixed Point:** Represents integers. The decimal point is always fixed at the end (rightmost position).
- **Floating Point:** Represents real numbers. The decimal/binary point "floats" based on the exponent value.

8.1 Sign and Magnitude Representation

The most significant (leftmost) bit is used as a **sign bit**:

- Sign bit = 0 → Positive number
- Sign bit = 1 → Negative number
- The remaining bits represent the actual magnitude (true binary)

Example (5-bit):

- $+12 = 01100$ (sign bit 0, magnitude 1100)
- $-12 = 11100$ (sign bit 1, magnitude 1100)

Drawbacks:

48. Addition and subtraction need to consider both sign and relative magnitude
49. Two representations of zero: +0 (00000) and -0 (10000)

8.2 One's Complement Representation

Negative values are obtained by **complementing each bit** of the corresponding positive number (inverting all bits).

Process: 1s complement of a number = subtract that number from $(2^n - 1)$, i.e., from 1111 for a 4-bit number

Example: 1s complement of 0101 = 1010

- $+12 = 01100$ (5-bit)
- $-12 = 10011$ (1s complement of 01100)

Drawback: Still two representations of zero: +0 (0000) and -0 (1111).

8.3 Two's Complement Representation

The **2s complement** of a number is obtained by adding 1 to its 1s complement. Equivalently, it equals 2^n minus the number.

Steps to find 2s complement:

50. Find the 1s complement (invert all bits)
51. Add 1 to the result

Example: 2s complement of 0101:

- 1s complement = 1010
- Add 1: $1010 + 1 = 1011$ → This is the 2s complement

Example:

- $+12 = 01100$

- $-12 = 10100$ (2s complement)

KEY RULE: In all three systems, the leftmost bit is 0 for positive numbers and 1 for negative numbers. 2s complement is the MOST WIDELY USED — it has a unique zero and arithmetic is simpler.

COMPUTER ORGANIZATION

For example 1s complement of 0101 is 1010. The process of forming the 1s complement of a given number is equivalent to subtracting that number from $2^n - 1$ i.e from 1111 for 4 bit number.

Two's Complement (2's) Representation Forming the 2's complement of a number is done by subtracting that number from 2^n . So 2's complement of a number is obtained by adding 1 to 1s complement of that number.

Ex: 2's complement of 0101 is $1010 + 1 = 1011$

NOTE: In all systems, the leftmost bit is 0 for positive number and 1 for negative number.



Signed integer represented in three ways.

Represent +12 and -12 in the following ways.

1. Sign and Magnitude Representation:

Sign bit True Magnitude (actual binary bits)

Ex: a. +12 is 01100 [4 bits are for representing number 12 and 1 bit is for sign. Hence]
b. -12 is 11100 [it is signed 5 bit number representation]

2. One's Complement (1's) Representation:

Sign bit 1's complement of actual binary bits

Ex: a. +12 is 01100 and b. -12 is 10011

3. Two's Complement (2's) Representation:

Sign bit 2's complement of actual binary bits

Ex: a. +12 is 01100 and b. -12 is 10100

Note: unsigned (+ve) number representation remains in same manner in all three ways.

Figure — Signed Integer Representations & 2's Complement Operations

COMPUTER ORGANIZATION

Binary, Signed-Integer Representations

B $b_3 b_2 b_1 b_0$	Values represented		
	Sign and magnitude	1's complement	2's complement
0 1 1 1	+7	+7	+7
0 1 1 0	+6	+6	+6
0 1 0 1	+5	+5	+5
0 1 0 0	+4	+4	+4
0 0 1 1	+3	+3	+3
0 0 1 0	+2	+2	+2
0 0 0 1	+1	+1	+1
0 0 0 0	+0	+0	+0
1 0 0 0	-0	-7	-6
1 0 0 1	-1	-6	-7
1 0 1 0	-2	-5	-6
1 0 1 1	-3	-4	-5
1 1 0 0	-4	-3	-4
1 1 0 1	-5	-2	-3
1 1 1 0	-6	-1	-2
1 1 1 1	-7	-0	-1

Figure 2.1. Binary, signed-integer representations.

2's-Complement Add and Subtract Operations

(a) $\begin{array}{r} 0010 \\ + 0011 \\ \hline \end{array} \begin{array}{l} (+2) \\ (+3) \end{array}$ (b) $\begin{array}{r} 0100 \\ + 1010 \\ \hline \end{array} \begin{array}{l} (+4) \\ (-6) \end{array}$

Figure 2.1 — Binary Signed Integer Representations Table

8.4 Overflow Conditions

Overflow occurs when the result of an arithmetic operation cannot be represented in the given number of bits.

Rule: Overflow occurs when the carry-IN to the sign bit (MSB) does NOT equal the carry-OUT from the sign bit.

Overflow occurs when:

52. Adding two positive numbers and the result appears negative (sign bit becomes 1)
53. Adding two negative numbers and the result appears positive

A simple detection rule: Overflow happens when BOTH operands have the SAME sign but the result has a DIFFERENT sign.

Examples:

- $(+5) + (+3) = +8 \rightarrow$ No overflow (result fits in range)

- $(+5) + (+5) = -6 \rightarrow \text{OVERFLOW!}$ (in 4-bit 2s complement, range is -8 to +7)
- $(-5) + (-3) = -8 \rightarrow \text{No overflow}$
- $(-5) + (-5) = +6 \rightarrow \text{OVERFLOW!}$

9. IEEE Standard for Floating Point Numbers

Floating-point numbers are so called because the decimal/binary point "**floats**" over the base depending on the exponent value. A floating point number consists of two components:

- **Exponent (E):** Determines the scale/magnitude
- **Mantissa/Significand (M or F):** Provides the precision/significant digits

Precision	Base	Sign Bits	Exponent Bits	Significand Bits
Single Precision (32-bit)	2	1	8	23+1 (implied 1)
Double Precision (64-bit)	2	1	11	52+1 (implied 1)

IEEE-754 32-bit Single-Precision

Bit layout (32 bits total):

| S (1) | Exponent E (8 bits) | Fraction F (23 bits) |

- **S (Bit 31):** Sign bit — 0 for positive, 1 for negative
- **E (Bits 30–23):** Exponent — stored in biased notation. Bias = 127 for single precision
- **F (Bits 22–0):** Fraction (Mantissa) — 23 bits; the leading 1 is implicit (normalized form: $1.M \times 2^E$)



IEEE-754 32-bit Single Precision Format

Conversion: Decimal to IEEE 754

Steps:

54. Convert the decimal number to binary
55. Normalize to the form $1.M \times 2^E$
56. Find Mantissa M (23 bits, pad with zeros if needed)
57. Find Exponent $E = E' + 127$ (add the bias)
58. Determine Sign bit S
59. Assemble: S | E (8 bits) | M (23 bits)

COMPUTER ORGANIZATION

Example 1: Convert (3.5)₁₀ to IEEE 32 bit floating point format

Solution:

1. Convert the given decimal number into its equivalent binary.
 $0.5 \times 2 = 1.0 \quad 1$
 $0.0 \times 2 = 0 \quad 0$ [not required to multiply with 0]
 $(3.5)_{10} = (11.1)_2$
2. Normalize the number [$1.M \times 2^{E'}$ format is used]
 1.11×2^1 (here $E' = 1$, the power of 2)
3. M = Mantissa = $110\ 0000\ 0000\ 0000\ 0000\ 0000$ [extra bits as mantissa is of 23 bits]
4. Exponent $E = E' + 127$
 $= 1 + 127 = (128)_{10} = (1000\ 0000)_2$
5. Sign bit $S = 0 \Rightarrow$ positive number
6. $0\ 1000\ 0000\ 110\ 0000\ 0000\ 0000\ 0000\ 0000$ sign, exponent and mantissa

Example 1: Suppose that IEEE-754 32-bit floating-point representation pattern is
 $0\ 1000\ 0000\ 110\ 0000\ 0000\ 0000\ 0000\ 0000$.
obtain its normalized form.

Sign bit $S = 0 \Rightarrow$ positive number
 $E = 1000\ 0000 = 128D$
 Use the format $1.M$ and convert binary to decimal.
 Fraction is 1.110 (with an implicit leading 1)
 $= 1 + 1 \times 2^{-1} + 1 \times 2^{-2} = 1.75D$
 Since the E is > 127 hence use the formula $E' = E - 127$
 The number is $+1.75 \times 2^{(128-127)} = +3.5D$

Example 2: Suppose that IEEE-754 32-bit floating-point representation pattern is
 $1\ 01111110\ 100\ 0000\ 0000\ 0000\ 0000\ 0000$. **obtain its normalized form.**
 Sign bit $S = 1 \Rightarrow$ negative number
 $E = 0111\ 1110 = 126D$ (in normalized form)
 Fraction is 1.10 (with an implicit leading 1) $= 1 + 2^{-1} = 1.5D$
 The number is $-1.5 \times 2^{(126-127)} = -0.75D$

Figure — IEEE 754 Conversion Examples

Conversion: IEEE 754 to Decimal**Steps:**

60. Read Sign bit S (0 = positive, 1 = negative)
61. Read Exponent E (8 bits) $\rightarrow$ find $E' = E - 127$
62. Read Fraction $F \rightarrow$ Significand = $1.F$ (implied leading 1)
63. Value = $(-1)^S \times 1.F \times 2^{(E-127)}$

Example: Pattern: $0\ 10000000\ 110000000000000000000000$

- $S = 0 \rightarrow$ positive
- $E = 10000000_2 = 128 \rightarrow E' = 128 - 127 = 1$
- $F = 1.11_2 = 1 + 0.5 + 0.25 = 1.75$
- Value = $+1.75 \times 2^1 = +3.5$

10. Memory Locations & Addresses

Memory consists of millions of storage cells. Each cell can store one bit (0 or 1). A group of n bits is called a **word** of information, and n is called the **word length** (typically 8 to 64 bits).

- **Byte:** A unit of 8 bits
- **Word:** A group of n bits; word length = 8 to 64 bits
- **Address Space:** If 2^k = number of addressable locations, then 2^k addresses constitute the address space. Example: 24-bit address $\rightarrow 2^{24} = 16$ MB address space

- Byte locations have addresses 0, 1, 2, ...
- If the word-length is 32 bits, successive words are located at addresses 0, 4, 8, ... with each word having 4 bytes.

BIG-ENDIAN & LITTLE-ENDIAN ASSIGNMENTS

- There are two ways in which byte-addresses are arranged (Figure 2.3).
 - 1) **Big-Endian:** Lower byte-addresses are used for the more significant bytes of the word.
 - 2) **Little-Endian:** Lower byte-addresses are used for the less significant bytes of the word.
- In both cases, byte-addresses 0, 4, 8, ... are taken as the addresses of successive words in the memory.

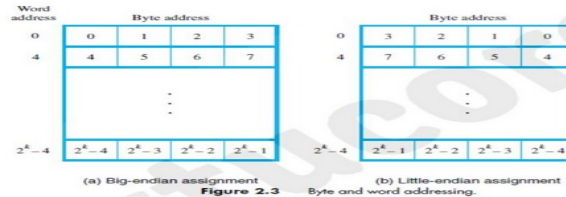


Figure 2.3 Byte and word addressing.

- Consider a 32-bit integer (in hex): 0x12345678 which consists of 4 bytes: 12, 34, 56, and 78.
 - Hence this integer will occupy 4 bytes in memory.
 - Assume, we store it at memory address starting 1000.
 - On little-endian, memory will look like

Address	Value
1000	78
1001	56
1002	34
1003	12

- On big-endian, memory will look like

Address	Value
1000	12
1001	34
1002	56
1003	78

WORD ALIGNMENT

- Words are said to be **Aligned** in memory if they begin at a byte-address that is a multiple of the number of bytes in a word.
- For example,
 - If the word length is 16(2 bytes), aligned words begin at byte-addresses 0, 2, 4, ...
 - If the word length is 64(2 bytes), aligned words begin at byte-addresses 0, 8, 16, ...
- Words are said to have **Unaligned Addresses**, if they begin at an arbitrary byte-address.

1-8

Figure 2.2 & 2.3 — Memory Words and Addressing

10.1 Byte Addressability

In **byte-addressable memory**, successive addresses refer to successive byte locations. Byte locations have addresses 0, 1, 2, ...

- If word length = 32 bits (4 bytes), successive words are at addresses 0, 4, 8, 12 ...
- Each word occupies 4 consecutive bytes

10.2 Big-Endian & Little-Endian Assignments

There are two ways byte addresses are arranged within a word:

64. **Big-Endian:** Lower byte-addresses are used for the MORE significant bytes. The MSB (most significant byte) is at the lowest address.
65. **Little-Endian:** Lower byte-addresses are used for the LESS significant bytes. The LSB (least significant byte) is at the lowest address.

Example: 32-bit hex integer 0x12345678, stored starting at address 1000:

Address	Big-Endian	Little-Endian
1000	12 (MSB)	78 (LSB)
1001	34	56
1002	56	34
1003	78 (LSB)	12 (MSB)

Memory Aid: Big-Endian = Big end (MSB) first. Little-Endian = Little end (LSB) first. Intel x86 = Little-Endian. Networking = Big-Endian.

10.3 Word Alignment

Words are said to be **Aligned** in memory if they begin at a byte-address that is a multiple of the number of bytes in a word.

- 16-bit word (2 bytes): aligned at byte-addresses 0, 2, 4, 6 ...
- 32-bit word (4 bytes): aligned at byte-addresses 0, 4, 8, 12 ...
- 64-bit word (8 bytes): aligned at byte-addresses 0, 8, 16, 24 ...

Unaligned Addresses: Words that begin at an arbitrary byte-address (not a multiple of word size). This requires extra memory accesses and is inefficient.

11. Memory Operations

Two basic memory operations are:

66. **Load (Read/Fetch):** Transfers a COPY of the contents of a specific memory location to the processor. Memory contents remain UNCHANGED.
67. **Store (Write):** Transfers information from a processor register to the specified memory location. This OVERWRITES (destroys) the original contents of that memory location.

Steps for Load Operation:

68. Processor sends the address of the desired location to the memory (via MAR)
69. Processor issues a "Read" signal to memory
70. Memory reads the data stored at that address
71. Memory sends the read data to the processor (via MDR)

Steps for Store Operation:

72. Processor sends the address of the memory location where it wants to store data (via MAR)
73. Processor issues a "Write" signal to memory
74. Content of register (MDR) is written into the specified memory location

Key Difference: Load is non-destructive (memory unchanged). Store is destructive (memory content replaced).

12. Instructions & Instruction Sequencing

A computer must have instructions capable of performing 4 types of operations:

75. **Data transfers:** Between memory and registers — MOV, PUSH, POP, XCHG, Load, Store
76. **Arithmetic & Logic operations:** ADD, SUB, MUL, DIV, AND, OR, NOT, XOR
77. **Program sequencing & control:** CALL, RET, LOOP, INT, JMP, Branch
78. **I/O transfers:** IN, OUT (for I/O-mapped I/O)

12.1 Register Transfer Notation (RTN)

RTN is used to formally describe register-level data transfers:

Location	Hardware Examples	RTN Example	Description
Memory	LOC, PLACE, NUM	$R1 \leftarrow [LOC]$	Load contents of memory location LOC into register R1
Processor	R0, R1, R2	$[R3] \leftarrow [R1] + [R2]$	Add R1 and R2, place sum in R3
I/O Register	DATAIN, DATAOUT	$R1 \leftarrow \text{DATAIN}$	Transfer contents of I/O register to R1

Square brackets [] denote "the contents of" the specified location.

12.2 Basic Instruction Types

Type	Syntax	Example	Description	Instructions for $C \leftarrow [A] + [B]$
Three Address	Opcode Src1, Src2, Dest	Add A,B,C	Add A and B, place result in C	1 instruction
Two Address	Opcode Src, Dest	Add A,B	Add A and B, result replaces B	Move B,C; Add A,C

One Address	Opcode Src/Dest	Load A; Add B; Store C	Uses implicit accumulator register	Load A; Add B; Store C
Zero Address	Opcode (no operands)	Push, Pop	Operands implicitly from stack	Not directly possible

Key Point: Access to data in registers is MUCH FASTER than to data in memory locations. Hence modern CPUs use many general-purpose registers.

12.3 Instruction Execution & Straight-Line Sequencing

The program is executed as follows:

79. Initially, the address of the first instruction is loaded into PC

80. The processor control circuits use the information in PC to fetch and execute instructions, one at a time, in the order of increasing addresses — this is called

Straight-Line Sequencing

81. During execution of each instruction, PC is incremented by 4 (for 32-bit word machines) to point to the next instruction

Two phases of Instruction Execution:

- **Fetch Phase:** The instruction is fetched from the memory location pointed to by PC and placed in the IR.
- **Execute Phase:** The contents of IR are examined (decoded) to determine which operation is to be performed. The specified operation is then performed by the processor.

12.4 Branching

For loops and conditional execution, we need **branching** — the ability to change the sequence of instruction execution.

- **Branch Instruction:** Loads a new value into the Program Counter (PC), causing the processor to fetch and execute the instruction at a new address — called the **Branch Target**
- **Conditional Branch:** Causes a branch ONLY if a specified condition is satisfied. If not satisfied, PC is incremented normally and execution continues sequentially.

Example: Adding n numbers using a loop:

- `Move N, R1` → R1 = counter (number of iterations)
- `Clear R0` → R0 = accumulator (sum = 0)
- `LOOP: [add next number to R0]`
- `Decrement R1` → R1 = R1 - 1
- `Branch>0 LOOP` → If R1 > 0, go back to LOOP
- `Move R0, SUM` → Store result

processor fetches and executes the instruction at this new address called the **Branch Target**.

- A **Conditional Branch Instruction** causes a branch only if a specified condition is satisfied. If the condition is not satisfied, the PC is incremented in the normal way, and the next instruction in sequential address order is fetched and executed.

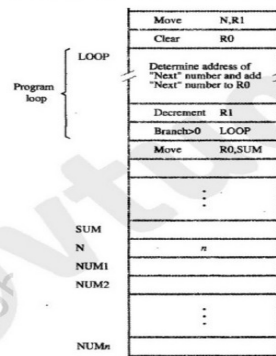


Figure 2.10 Using a loop to add n numbers.

CONDITION CODES

- The processor keeps track of information about the results of various operations. This is accomplished by recording the required information in individual bits, called **Condition Code Flags**.
- These flags are grouped together in a special processor-register called the condition code register (or status register).
- Four commonly used flags are:

Figure 2.10 — Using a Loop to Add n Numbers

12.5 Condition Codes

The processor keeps track of results of operations using individual bits called **Condition Code Flags**, grouped in the **Condition Code Register** (also called Status Register or Flags Register).

Flag	Name	Set to 1 When...
N	Negative	The result of the operation is negative
Z	Zero	The result of the operation is zero
V	Overflow	Arithmetic overflow occurs
C	Carry	A carry-out results from the operation (from the MSB)

These flags are tested by conditional branch instructions. E.g., Branch>0 checks the N and Z flags to decide whether to branch.

13. Solved Problems – Part 2

Problem 1: 2's Complement Operations

Perform the following in 2's complement 4-bit representation:

(a) **(+4) + (+3):**

```

0100 (+4)
+ 0011 (+3)
= 0111 (+7) ✓ No overflow

```

(b) **(-4) + (-3):**

```

1100 (-4)
+ 1101 (-3)
= 11001 → ignore carry: 1001 (-7) ✓ No overflow

```

(c) **(+5) + (+3) = +8 → OVERFLOW (max is +7 in 4-bit 2s complement)!**

```

0101 (+5)
+ 0011 (+3)
= 1000 → appears as -8 → OVERFLOW ✗

```

Problem 2: IEEE 754 Conversion — Decimal to Binary

Convert $(3.5)_{10}$ to IEEE 754 32-bit single precision:

82. **Binary:** $3.5 = 11.1_2$

83. **Normalize:** 1.11×2^1 ($E' = 1$)

84. **Mantissa M:** 110 0000 0000 0000 0000 0000 (23 bits, pad with zeros)

85. **Exponent E:** $E = E' + 127 = 1 + 127 = 128 = 1000\ 0000_2$

86. **Sign S:** $S = 0$ (positive)

87. **Result:** 0 10000000 11000000000000000000000

Problem 3: IEEE 754 — Binary to Decimal

Find the decimal value of: 1 01111110 100000000000000000000000

88. **S = 1:** negative number

89. **E = 01111110₂ = 126:** $E' = 126 - 127 = -1$

90. **Fraction:** $1.1_2 = 1 + 2^{-1} = 1.5$

91. **Value:** $-1.5 \times 2^{-1} = -0.75$

Problem 4: Memory Addressing

A computer has a 24-bit address bus. How many memory locations can be addressed? What is the maximum memory?

Solution:

- Number of addressable locations = $2^{24} = 16,777,216 = 16\text{M}$ locations
- If each location stores 1 byte: Maximum memory = 16 MB

Problem 5: Big-Endian vs Little-Endian

Store the 32-bit integer 0xABCD1234 starting at memory address 2000:

Address	Big-Endian (Value)	Little-Endian (Value)
2000	AB	34
2001	CD	12
2002	12	CD
2003	34	AB

14. Quick Revision Summary

All key formulas, definitions and facts at a glance — vtu-easy.com

Part 1: Basic Structure — Key Points

Topic	Key Fact / Formula
Computer Architecture	CA = Hardware + ISA + Organization
Functional Units (5)	Input, Memory, ALU, Output, Control Unit
Primary Memory	RAM, ROM, Cache — directly accessed by CPU, stores running programs
Secondary Memory	HDD, SSD, CD — long-term storage, not directly accessible by CPU
ALU Operations	Arithmetic (+, -, ×, ÷) and Logical (AND, OR, NOT, XOR)
Control Unit Role	Generates timing signals — does NOT process data
PC (Program Counter)	Holds address of NEXT instruction to fetch
IR (Instruction Register)	Holds CURRENT instruction being decoded/executed
MAR (Memory Address Reg)	Holds address of memory location to access
MDR (Memory Data Reg)	Holds data read from / to be written to memory
Single Bus	One transfer at a time — low cost, but bottleneck
Multiple Bus	Concurrent transfers — better performance, higher cost
Memory Mapped I/O	I/O registers treated as memory locations — use Load/Store
Processor Clock	$R = 1/P$ (Clock Rate = 1 / Clock Period)
Basic Performance Eq.	$T = (N \times S) / R$
RISC	Few simple fixed-format instructions, hardwired, highly pipelined
CISC	Many complex variable-format instructions, microprogrammed
Cache Memory	Copy of recently used instructions kept near CPU for fast access

Part 2: Machine Instructions — Key Points

Topic	Key Fact / Formula
Sign & Magnitude	MSB = sign (0:+, 1:-); Two zeros: +0 and -0
1s Complement	Negate = invert all bits; Two zeros still exist
2s Complement	Negate = invert all bits + 1; Unique zero; Used in all modern CPUs
Overflow (2s comp)	Carry-in to MSB $\neq$ Carry-out from MSB $\rightarrow$ Overflow
IEEE 754 Single	32 bits: 1 sign + 8 exponent + 23 mantissa; Bias = 127
IEEE 754 Double	64 bits: 1 sign + 11 exponent + 52 mantissa; Bias = 1023
Normalized Form	$1.M \times 2^E$ (leading 1 is implicit — not stored)
Stored Exponent	$E = E' + 127$ (for single precision)

Byte Addressability	Each byte has unique address; 32-bit word = 4 bytes = addresses 0,4,8...
Big-Endian	MSB stored at LOWEST address (most significant byte first)
Little-Endian	LSB stored at LOWEST address (least significant byte first)
Word Alignment	Word starts at address multiple of word size in bytes
Load (Read)	Non-destructive: copies memory → register; memory unchanged
Store (Write)	Destructive: copies register → memory; overwrites memory
3-Address Instruction	Two sources + one destination; $C = A \text{ op } B$ in one instruction
Condition Code Flags	N (negative), Z (zero), V (overflow), C (carry)
Straight-Line Sequence	PC increments by word size after each fetch — sequential execution
Branch Instruction	Loads new value into PC — changes execution sequence
Conditional Branch	Branches ONLY if specified condition (flag) is satisfied

Frequently Asked VTU Questions

Question (Marks)	Key Answer Points
Explain functional units (6M)	All 5 units with roles + Figure 1.1 diagram
Explain operation of computer (10M)	5 units + instruction cycle + PC/MAR/MDR/IR roles
Discuss PC, MAR, MDR, IR (8M)	4 registers + 10 steps of instruction execution
Single bus vs multiple bus (6M)	Diagrams + advantages + disadvantages of each
Explain OS routine sharing (8M)	OS tasks + timeline t_0 to t_5 + diagram
Basic performance equation (4M)	$T = N \times S / R$ + explain N, S, R + how to reduce T
CISC vs RISC table (6M)	Full comparison table with 7 features
Sign, 1s, 2s complement	Representations + examples + drawbacks
IEEE 754 format (8M)	Format + steps for conversion both ways + examples
Big vs Little Endian (4M)	Definition + example with 0x12345678
Memory operations (4M)	Load (non-destructive) + Store (destructive) + steps
Instruction types (6M)	All 4 types: 3-addr, 2-addr, 1-addr, 0-addr with examples
Branching and condition codes	Branch target + conditional branch + N,Z,V,C flags